BACKEND – LAB 1 – MIGRATIONS, SEEDERS Y MODELS

MIGRATIONS

En la carpeta *database/migrations* es donde se crean todas las tablas. Esto se verá reflejado en *src/models/*

A partir del diagrama UML se crean las tablas. Así comienzan todas las tablas:

```
module.exports = {
  up: async (queryInterface, Sequelize) => {
    await queryInterface.createTable('Products', {
```

1. Cabecera creación tablas, /src/database/migrations/

Para cerrar una tabla debemos poner lo siguiente dentro de los atributos para que se especifique si se ha **creado** o **actualizado** una tabla y un **"cierre"**:

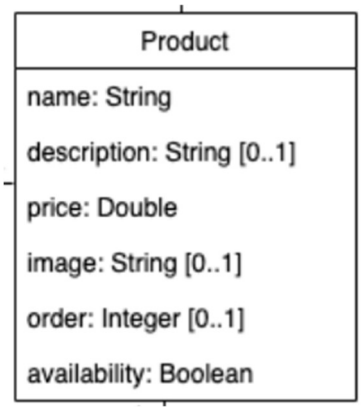
```
createdAt: {
  allowNull: false,
  type: Sequelize.DATE,
  defaultValue: new Date()
},
```

```
updatedAt: {
  allowNull: false,
  type: Sequelize.DATE,
  defaultValue: new Date()
}
```

```
    })
  },
  down: async (queryInterface, Sequelize) => {
    await queryInterface.dropTable('Restaurants')
  }
}
```

En la tabla tenemos que poner de cada **atributo**: su *nombre*, su *type* y *allowNull* (si son opcionales). También se pueden añadir *defaultValue* (valor por defecto).

```
name: {
  allowNull: false,
  type: Sequelize.STRING
},
description: {
  type: Sequelize.TEXT
},
price: {
  allowNull: false,
  type: Sequelize.DOUBLE
},
```



2. Ejemplo atributos de tablas, /src/database/migrations/

Lo primero que se pone es la **PrimaryKey** que no sale en los diagramas pero <u>cada tabla debe tenerla</u>.

```
id: {
  allowNull: false,
  autoIncrement: true,
  primaryKey: true,
  type: Sequelize.INTEGER
},
```

3. PrimaryKey de tablas, /src/database/migrations/

Para poner las **ForeingKey**, se escriben las Id de las tablas relacionadas, se puede ver en *src/models/* las claves foráneas que tiene cada tabla. Por ejemplo, la tabla *'Products'* tiene su propia PK (id) y una FK que proviene de la tabla *'ProductCategories'*, este nombre sale de la cabecera de *'create-productCategory'*. Dentro de la descripción de los atributos añadiremos la FK, al igual que la PK y los atributos, pero <u>es necesario referenciar de qué tabla viene.</u>

```
productCategoryId: {
  type: Sequelize.INTEGER,
  allowNull: false,
  references: {
    model: {
      tableName: 'ProductCategories'
    },
    key: 'id'
  }
},
```

4. Ejemplo de Foreing Key en creación de tablas, /src/database/migrations/

Hay algunas clases hijas en las que tenemos que especificar qué pasa cuando la clase padre se borra.

onDelete: 'cascade' → si se borra la ForeingKey (padre) , <u>se borran también sus hijos</u> que tengan '*cascade*'.

```
restaurantId: {
  type: Sequelize.INTEGER,
  allowNull: false,
  references: {
    model: {
      tableName: 'Restaurants'
    },
    key: 'id'
  },
  onDelete: 'cascade'
},
```

En la tabla *create-product* tenemos la entidad *'Product'*, con su PK '*productId'* y su FK '*restaurantId'* porque esos productos pertenecen al restaurante. Al definir la FK '*restaurantId'* como atributo de '*Product'*, si ponemos *onDelete: 'cascade'* y posteriormente borramos un restaurante con esa restaurantId, también se borraran todos los productos cuya FK sea esa '*restaurantId'*, ya que esos productos pertenecen a ese restaurante.

Si pusiéramos **onDelete: 'set null'** , al borrar la clase padre (la FK), <u>las clases hijas no se borrarían, sino que se quedarían huérfanas</u>. Con el valor *null* en el campo de la FK. Es importante que entonces este atributo FK tenga ***allowNull: true***.

Si no se pone onDelete y se intenta borrar una clase padre da ERROR, y tendrías que eliminar manualmente todas las clases hijas. para poder borrar la clase padre.

Las **propiedades derivadas** vendrán dadas en el UML de la siguiente forma. Estas no se deben escribir en las tablas. Se definirán más adelante en *src/models*.

/ averageServiceMinutes: Double [0..1]

5. Propiedad derivada, entidad Restaurant

Los **ENUM** se escriben de la siguiente manera:

```
status:{
  allowNull:false,
  type: Sequelize.ENUM('online', 'offline', 'closed', 'temporarily closed')
},
```

6. Ejemplo de ENUM

PARA RUNEAR LAS MIGRACIONES:

```
cd DeliverUS-Backend
```

Y luego ejecuta:

```
npx sequelize-cli db:migrate
```

Después de hacer esto, deberías encontrar las tablas creadas en tu mariadb.

Para deshacer migraciones puedes ejecutar:

```
npx sequelize-cli db:migrate:undo:all
```

orderproducts	0	128,0 KiB	2026-02-18 10:10:06		InnoDB	Table
orders	0	192,0 KiB	2026-02-18 10:10:06		InnoDB	Table
productcatego...	6	64,0 KiB	2026-02-18 10:10:06		InnoDB	Table
products	203	192,0 KiB	2026-02-18 10:10:06		InnoDB	Table
restaurantcate...	2	64,0 KiB	2026-02-18 10:10:06		InnoDB	Table
restaurants	7	192,0 KiB	2026-02-18 10:10:06		InnoDB	Table
sequelizemeta	6	128,0 KiB	2026-02-18 10:10:04	2026-03-10 18:00:30	InnoDB	Table
users	2	192,0 KiB	2026-02-18 10:10:06		InnoDB	Table

7. HeidiSQL al realizar 'npx sequelize-cli db:migrate'

SEEDERS

Los archivos semilla se utilizan para llenar las tablas de la base de datos con datos de muestra o de prueba. Puede encontrarlos en la `seeders` carpeta . Tenga en cuenta que `restaurants_seeder.js` se asume un nombre específico para los campos de la tabla de restaurantes.

Puede ejecutar sus sembradores para completar la base de datos ejecutando:

```
npx sequelize-cli db:seed:all
```

Y puedes deshacerlos ejecutando:

```
npx sequelize-cli db:seed:undo:all
```

Puede encontrar más información sobre los sembradores en:
https://sequelize.org/master/manual/migrations.html#creating-the-first-seed

---

Si realiza algún cambio en las migraciones o los sembradores, puede actualizar la base de datos ejecutando los comandos "deshacer migraciones", "ejecutar migraciones" y "ejecutar sembradores" simultáneamente; para ello, abra una terminal y ejecute "<nombre del script>" `npm run migrate:backend`. Este script está configurado para ejecutarse en el directorio raíz.

8. Cómo ejecutar las seeders

root | Base de datos: deliverus | Tabla: restaurants | Datos | Consulta

deliverus.restaurants: 7 filas en total (exacto) — Siguientes | Mostrar todo | Ordenación

#	1 id	2 name	3 description	4 address	5 postalCo...	6 url
1	1	Casa Félix	Cocina Tradicional	Av. Reina Mercedes 51, Sevilla	41012	https://goo.gl/maps/GZEfzge4zXz...
2	2	100 montaditos	A fun and varied way to enjoy fo...	Józefa 34, Kraków, Poland	123-4567	http://spain.100montaditos.com/
3	3	0 products	0 products	0 products address	123-4567	http://0products.com/
4	4	1 product	1 product	1 products address	123-4567	http://1products.com/
5	5	30 products	30 products	30 products address	123-4567	http://30products.com/
6	6	50 products	50 products	50 products address	123-4567	http://50products.com/
7	7	100 products	100 products	100 products address	123-4567	http://100products.com/

9. HeidiSQL al ejecutar 'npx sequelize-cli db:seed:all'

MODELS

Cada modelo es una clase que lleva el nombre de su tabla correspondiente (pero en singular) y extiende la clase Modelo de Sequelize.

Sus cabeceras son:

```
import { Model } from 'sequelize'
import moment from 'moment'

const loadModel = (sequelize, DataTypes) => {
  class Restaurant extends Model {
```

10. Cabecera Model, /src/models/

Dentro se describen las relaciones que tiene con el resto de clases. Estas relaciones podemos verlas en el modelado conceptual (diagrama UML).

Observe que también hemos definido las relaciones entre los modelos. Por ejemplo, el modelo '*Restaurant*' está relacionado con '*RestaurantCategory*', '*User*', '*Product*' y '*Order*. Para definir estas relaciones, debemos incluir el método **associate**:

```
static associate (models) {
  // define association here
  Restaurant.belongsTo(models.RestaurantCategory, { foreignKey: 'restaurantCategoryId', as: 'restaurantCategory' })
  Restaurant.belongsTo(models.User, { foreignKey: 'userId', as: 'user' })
  Restaurant.hasMany(models.Product, { foreignKey: 'restaurantId', as: 'products' })
  Restaurant.hasMany(models.Order, { foreignKey: 'restaurantId', as: 'orders' })
}
```

11. Relaciones clase 'Restaurant'

Otra relación que existe es la de la **clase asociada**, como por ejemplo, '*OrderProduct*' que es una clase asociada entre '*Product*' y '*Order*'. En ambos modelos se debe incluir este fragmento de código dentro del método **associate**:

```
9     static associate (models) {
10      // define association here
11      const OrderProducts = sequelize.define('OrderProducts', {
12        quantity: DataTypes.INTEGER,
13        unityPrice: DataTypes.DOUBLE
14      })
15
16    Ask for Edits Ctrl+I | sTo(models.Restaurant, { foreignKey: 'restaurantId', as: 'restaurant', onDelete: 'cascade' })
17      Product.belongsTo(models.ProductCategory, { foreignKey: 'productCategoryId', as: 'productCategory' })
18      Product.belongsToMany(models.Order, { as: 'orders', through: OrderProducts })
19    }
```

12. Clase asociada dentro del model

Aquí dentro se está creando la tabla '*OrderProducts*' por lo que no será necesario crear la migration *create-order-product* ni el model '*OrderProducts*'. Tampoco tenemos que poner nada en *create-product* ni *create-order* sobre la clase asociada, solo en el modelo.

Sobre **onDelete**, ya vimos que se pone en el atributo de la FK en el create. Además tendremos que ponerlo también el modelo, donde se dice la relación entre las tablas. *[línea 16, Imagen 12.]*

También se definen los métodos para calcular las **propiedades derivadas**. Vemos el ejemplo de la clase '*Restaurant*': */averageServiceTime*

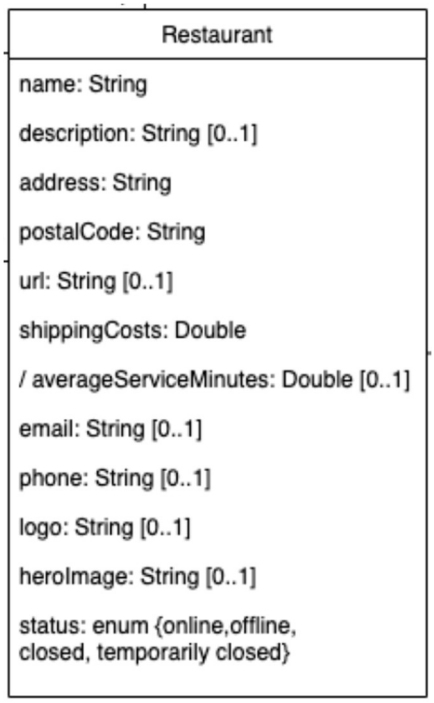
/ averageServiceMinutes: Double [0..1]

```
async getAverageServiceTime () {
  try {
    const orders = await this.getOrders()
    const serviceTimes = orders.filter(o => o.deliveredAt).map(o => moment(o.deliveredAt).diff(moment(o.createdAt), 'minutes'))
    return serviceTimes.reduce((acc, serviceTime) => acc + serviceTime, 0) / serviceTimes.length
  } catch (err) {
    return err
  }
}
```

13. Definición del método getAverageServiceTime()

Finalmente, se declaran todos los atributos que pusimos en create-...

```
Restaurant.init({
  name: DataTypes.STRING,
  description: DataTypes.STRING,
  address: DataTypes.STRING,
  postalCode: DataTypes.STRING,
  url: DataTypes.STRING,
  shippingCosts: DataTypes.DOUBLE,
  email:DataTypes.STRING,
  logo: DataTypes.STRING,
  heroImage: DataTypes.STRING,
  status: DataTypes.ENUM,
  restaurantCategory: DataTypes.INTEGER,
  userId: DataTypes.INTEGER
  // TODO: creo que ya está
}, {
  sequelize,
  modelName: 'Restaurant'
})
return Restaurant
```



BACKEND – LAB 2 – ROUTES, CONTROLLERS

ROUTES

Aquí se definen las URI: la **ruta o dirección** del recurso con el que quieres interactuar. *"Cuando se recibe una petición en una **Route**, se ejecutan los **Middlewares** y **Validations**; si la petición supera todos estos filtros, el **Controller** ejecuta su funcionalidad."*

(Middlewares y las Validations [página 11]). En la *imagen 13* sólo hay controllers, aún no están los Middlewares y Validations.

```
DeliverUS-Backend > src > routes > JS RestaurantRoutes.js > ...
1   import RestaurantController from '../controllers/RestaurantController.js'
2
3   const loadFileRoutes = function (app) {
4
5     app.route('/restaurants')
6     .get(RestaurantController.index)
7     .post(RestaurantController.create)
8
9     app.route('/restaurants/:restaurantId')
10    .get(RestaurantController.show)
11    .put(RestaurantController.update)
12    .delete(RestaurantController.destroy)
13  }
14  export default loadFileRoutes
```

14. Ejemplo de Route solo con controllers

Peticiones HTTP relacionadas con cada método:

GET → .index o .show - POST → .create - PUT → .update - DELETE → .destroy

Dentro de las Routes tenemos que poner cada URI y qué tiene que pasar al pinchar ahí. Aquí por ejemplo tenemos que, al estar en *restaurants* podemos hacer **GET** y **la ruta llamará al método .index del '*RestaurantController*'.**

Vemos que en */restaurants* el GET llama al método *.index* mientras que en */restaurants/:restaurantId* el GET llama al método ***.show***. <u>Esto es porque *.index* devuelve un ARRAY de todos los restaurantes mientras que *.show* devuelve solo el restaurante que se pide en concreto con el Id.</u>

CONTROLLERS

Es donde se "definen" todas las funciones que se utilizarán como métodos desde Routes: .index .create .show .update y .destroy

CABECERA:

```
DeliverUS-Backend > src > controllers > RestaurantController.js > index
1  import { Restaurant, Product, RestaurantCategory, ProductCategory } from '../models/models.js'
```

15. Cabecera de RestaurantController

Al final de cada controller hay que crear el objeto ***NombreController*** con las funciones en su interior para ser llamados desde las rutas como métodos.

```
91  const RestaurantController = {
92    index,
93    create,
94    show,
95    update,
96    destroy
97  }
98  export default RestaurantController
```

16. Creación de la constante RestaurantController con sus funciones

Aquí tenemos todas las funciones. Código explicado en las imágenes.

INDEX:

```
3  const index = async function (req, res) { // index devuelve TODOS los restaurantes
4    try {
5      const restaurants = await Restaurant.findAll( // llama al modelo Restaurant y .findAll para que los muestre todos
6        {
7          attributes: { exclude: ['userId'] },  //trae toda la info EXCLUYENDO la id del propietario
8          include:  // INCLUYE también la RestaurantCategory que viene de otra tabla (hace un JOIN)
9        {
10         model: RestaurantCategory,
11         as: 'restaurantCategory'    //le pone un 'alias'
12       },
13         order: [[{ model: RestaurantCategory, as: 'restaurantCategory' }, 'name', 'ASC']]
14         //como ordena la lista antes de devolverla, aquí le dice que por NAME en orden ASCendente
15       }
16     )
17     res.json(restaurants) // devuelve al cliente 200 OK
18   } catch (err) {
19     res.status(500).send(err) //ERROR
20   }
21 }
```

17. Función index de RestaurantController

CREATE:

```
24 const create = async function (req, res) {  //declaro la función create, asincrona, request/response
25   const newRestaurant = Restaurant.build(req.body)  // construye un 'borrador' del nuevo restaurant con la info de req.body
26   try {   //try/catch
27     const restaurant = await newRestaurant.save() //.save hace INSERT en la base de datos del req.body
28                          // await para decirle que espere confirmacion de la base de datos para seguir
29     res.json(restaurant)  // devuelve al cliente 200 OK
30   } catch (err) { //try/catch, si el try no sale bien, salta el error del catch
31     res.status(500).send(err)   //responde al cliente que ERROR
32   }
33 }
```

18. Función create de RestaurantController

SHOW:

```
const show = async function (req, res) {
  // es como el index, pero solo busca el restaurante concreto por la Id
  // .findByPk
  try{
    const restaurant = await Restaurant.findByPk(req.params.restaurantId, {
```

19. Diferencia entre función show e index de RestaurantController

UPDATE:

```
60  const update = async function (req, res) {    //actualizar
61    try {
62      await Restaurant.update(req.body, { where: { id: req.params.restaurantId } })
63      // recibe el req.body con los datos nuevos
64      // where:{...} --> y aplicalos solamente al restaurante que coincida con el id
65
66      const updatedRestaurant = await Restaurant.findByPk(req.params.restaurantId)
67      // esta línea es saltable
68      // para que se muestren los nuevos datos actualizados sin tener que realizar un GET
69
70      res.json(updatedRestaurant)
71    } catch (err) {
72      res.status(500).send(err)
73    }
74  }
```

20. Función update de RestaurantController

DESTROY:

Funciona igual que *.update*, pero este elimina.

```
76  const destroy = async function (req, res) {
77    try {
78      const result = await Restaurant.destroy({ where: { id: req.params.restaurantId } })
79      let message = ''    //crea variable message vacia
80      if (result === 1) {
81        message = 'Sucessfuly deleted restaurant id.' + req.params.restaurantId
82      } else {
83        message = 'Could not delete restaurant.'
84      }
85      res.json(message)
86    } catch (err) {
87      res.status(500).send(err)
88    }
89  }
```

21. Función destroy de RestaurantController

BACKEND – LAB 3 – VALIDATION & MIDDLEWARE

MIDDLEWARE

Los **middlewares** sirven para comprobar si los datos que llegan en una solicitud cumplen con los requisitos de información.

Se definen en ficheros.js y se guardan en *src/middlewares*. Estas funciones serán llamadas desde **Routes**.

"Si alguien viene por la URL /restaurants con el método POST, primero pásalo por estos guardias (middlewares) y, si todo está bien, llévalo con el chef (RestaurantController.create)".

Los middlewares que tenemos definidos son:

isLoggedIn → comprueba si el usuario ha iniciado sesión.

hasRole('rol') → recibe una lista de roles y comprueba si el usuario que a iniciado sesión tiene el rol necesario.

checkEntityExists → comprueba para un ID y entidad si existe un registro en la tabla correspondiente de la base de datos que coincida con ese ID.

handleValidation → se pone después de haber llamado a algún Validation y si ese validation no se cumple, hace que salte el error.

Otros middlewares que encontramos son:

1. Middlewares Globales

- `loadGlobalMiddlewares` : Se usa en JS **src/app.js** nada más arrancar la aplicación. Sirve para cargar configuraciones generales de Express (como permitir que el servidor entienda datos en formato JSON, o configurar CORS para que el frontend pueda conectarse).

2. Middlewares de Restaurantes (JS RestaurantRoutes.js)

- `checkRestaurantOwnership` : Se usa antes de editar o borrar un restaurante, o de ver sus analíticas/pedidos. Sirve para asegurar que el usuario que intenta hacer eso es realmente el *owner* (propietario) de ese restaurante en concreto.
- `restaurantHasNoOrders` : Se usa antes de *borrar* un restaurante (`DELETE`). Impide que un dueño borre un restaurante si este ya tiene pedidos registrados en el sistema (para no romper el historial de compras).
- `checkRestaurantExists` : Se usa en validaciones (ej. al crear un producto nuevo) para confirmar que el ID del restaurante al que quieres añadir el producto realmente existe en la base de datos.

3. Middlewares de Productos (JS ProductRoutes.js)

- `checkProductOwnership` : Antes de editar o borrar un producto, confirma que sea tuyo.
- `checkProductRestaurantOwnership` : Al crear un nuevo producto para un restaurante, verifica que tú seas el dueño de ese restaurante.
- `checkProductHasNotBeenOrdered` : Antes de borrar o editar el precio de un producto, verifica que nadie lo haya pedido ya. Si ya está en un pedido pagado, el sistema no te deja borrarlo o cambiarle el precio porque estropearía la factura del cliente.

4. Middlewares de Pedidos (JS OrderRoutes.js)

- `checkOrderOwnership` : Para los dueños de restaurante. Asegura que el pedido que el dueño intenta modificar (ej. enviarlo) pertenece a su restaurante.
- `checkOrderCustomer` : Para los clientes. Asegura que el pedido que intentan ver es suyo y no el de otro cliente.
- `checkOrderVisible` : Combina los dos anteriores: permite ver el pedido si eres el cliente que lo hizo, o si eres el dueño del restaurante que lo prepara.
- `checkOrderIsPending` : (Control de estado) Antes de poder editar un pedido o cancelarlo, comprueba que siga "Pendiente". Si el restaurante ya lo está cocinando, ya no puedes editarlo.
- `checkOrderCanBeSent` : Asegura que un pedido solo pueda cambiar a estado "Enviado" si su estado actual era el correcto para dar ese paso.
- `checkOrderCanBeDelivered` : Asegura que un pedido solo pueda marcarse como "Entregado" si previamente estaba como "Enviado".

22. Explicación del resto de middlewares que encontramos en el proyecto según Gémini

La estructura final que quedaría de una Route sería:

app.route('ruta')

.VERBO_HTTP (

Middlewares & Validations,

Llamada al controller.funcion)

Vemos cómo se llaman a los middlewares y validations dentro de Routes:

```
app.route('/restaurants/:restaurantId')
  .get(
    checkEntityExists(Restaurant, 'restaurantId'),
    RestaurantController.show)
  .put(
    isLoggedIn,
    hasRole('owner'),
    checkEntityExists(Restaurant, 'restaurantId'),
    RestaurantMiddleware.checkRestaurantOwnership,
    handleFilesUpload(['logo', 'heroImage'], process.env.RESTAURANTS_FOLDER),
    RestaurantValidation.update,
    handleValidation,
    RestaurantController.update)
  .delete(
    isLoggedIn,
    hasRole('owner'),
    checkEntityExists(Restaurant, 'restaurantId'),
    RestaurantMiddleware.restaurantHasNoOrders,
    RestaurantMiddleware.checkRestaurantOwnership,
    RestaurantController.destroy)
```

23. Ejemplo RestaurantRoutes /restaurants/:restaurantId

VALIDATION

En el fichero '*RestaurantValidation*' vamos a verificar que lo que proporciona el cliente está en el formato necesario. Es importante importar el método ***check***.

```
import { check } from 'express-validator'
import { checkFileIsImage, checkFileMaxSize } from './FileValidationHelper.js'
const maxFileSize = 2000000 // around 2Mb
```

```
DeliverUS-Backend > src > controllers > validation > JS RestaurantValidation.js > update > custom() callback
const create = [
  // al poner check('xxxx') apunta a ese campo 'xxxx' introducido por el cliente
  // comprueba que se cumplan todas las 'condiciones de formato'
  check('name').exists().isString().isLength({ min: 1, max: 255 }).trim(),
  // hay un campo name, es un String, de tamaño 1 - 255, (.trim limpia de espacios innecesarios)
  check('description').optional({ nullable: true, checkFalsy: true }).isString().trim(),
  // hay un campo descripcion, permite opcional --> {que sea null o que sea false}, es un String, .trim
  check('address').exists().isString().isLength({ min: 1, max: 255 }).trim(),
  check('postalCode').exists().isString().isLength({ min: 1, max: 255 }),
  check('url').optional({ nullable: true, checkFalsy: true }).isString().isURL().trim(),
  // Comprueba que el texto tenga una estructura de dirección web válida
  // (que incluya cosas como http://, https:// o un dominio como .com, .es).
  check('shippingCosts').exists().isFloat({ min: 0 }).toFloat(),
  // .isFloat: comprueba que sea de tipo Double
  // .toFloat: si es un texto "3.95" lo convierte al tipo
  check('email').optional({ nullable: true, checkFalsy: true }).isString().isEmail().trim(),
  check('phone').optional({ nullable: true, checkFalsy: true }).isString().isLength({ min: 1, max: 255 }).trim(),
  check('restaurantCategoryId').exists({ checkNull: true }).isInt({ min: 1 }).toInt(),
  check('userId').not().exists(),

  // creamos un check personalizado con el .custom()
  // value == 'imagen.jpeg' - req == lleva l a peticion del cliente
  check('heroImage').custom((value, { req }) => {
    // si se cumple lo devuelve, (checkFileIsImage es una funcion creada en FileValidationHelper.js)
    return checkFileIsImage(req, 'heroImage')
      // si da error devuelve este mensaje:
  }).withMessage('Please upload an image with format (jpeg, png).'),
  check('heroImage').custom((value, { req }) => {
    return checkFileMaxSize(req, 'heroImage', maxFileSize)
  }).withMessage('Maximum file size of ' + maxFileSize / 1000000 + 'MB'),
  check('logo').custom((value, { req }) => {
    return checkFileIsImage(req, 'logo')
  }).withMessage('Please upload an image with format (jpeg, png).'),
  check('logo').custom((value, { req }) => {
    return checkFileMaxSize(req, 'logo', maxFileSize)
  }).withMessage('Maximum file size of ' + maxFileSize / 1000000 + 'MB')
]
```